Day 7 — Final Project: Build Your Own Copilot

> 🎯 **Goal**: Produce a Senior-level, production-quality agentic system.

> ⏱️ **Time**: Full day (6–8h)

> 📦 **Deliverable**: A full, working Agent system with planning, collaboration, RAG, and tools

---

Table of Contents

1. Project Overview
2. Project A: Engineering Copilot
3. Project B: DevOps Optimization Copilot
4. Evaluation Criteria
5. What Makes This Senior-Level

Project Overview

Today you integrate everything from Days 1–6 into a production-quality system. Choose **one** of two projects:

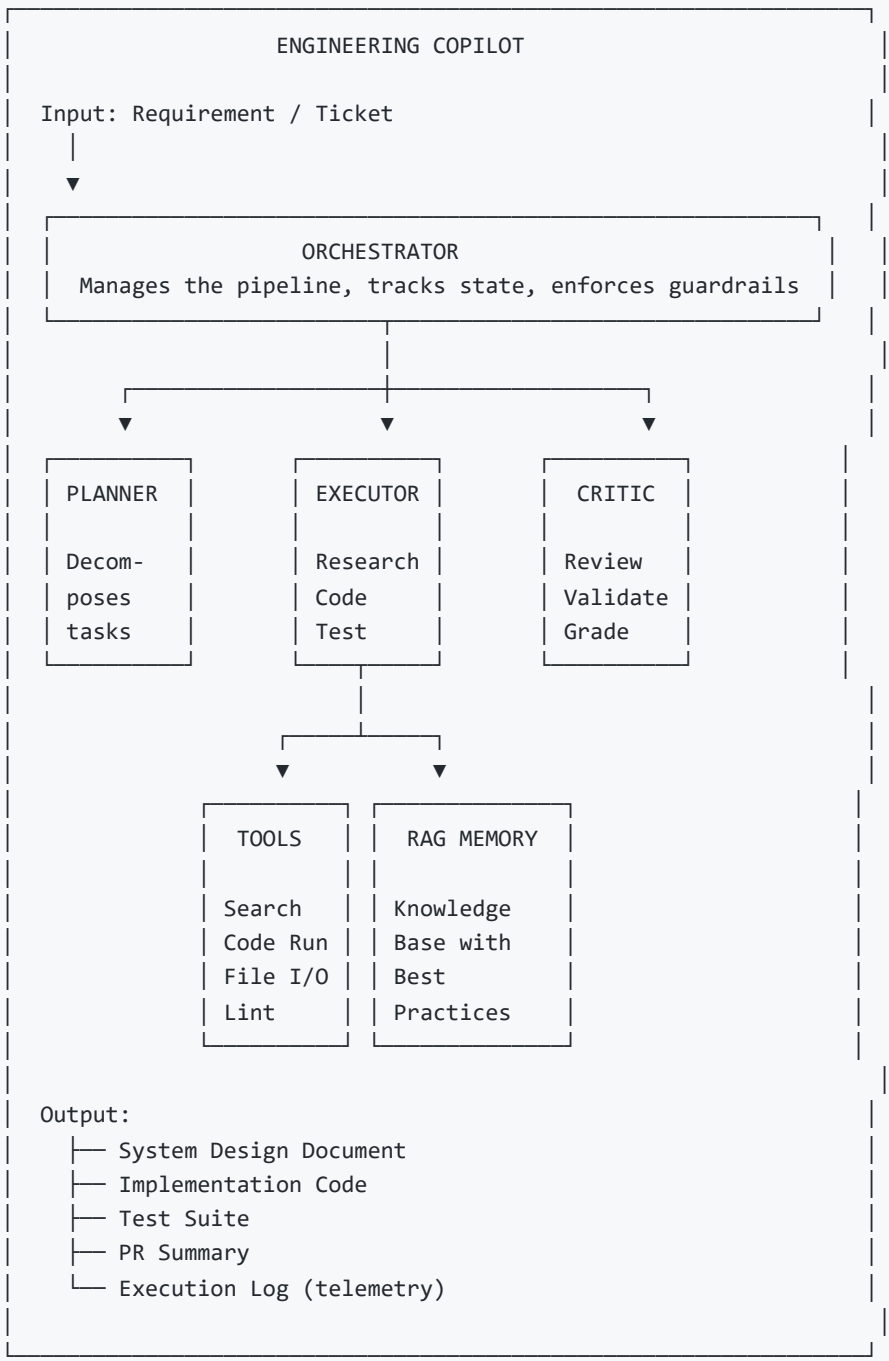
```
┌─────────────────────────────────────────────────────────────┐
│                    CHOOSE YOUR PROJECT                      │
│                                                             │
│  🅰 ENGINEERING COPILOT (Recommended)                       │
│     Input: A requirement or ticket                          │
│     Output: System design + Code + Tests + PR summary       │
│     Skills: Planning, coding, RAG, multi-agent, tools       │
│                                                             │
│  🅱 DEVOPS OPTIMIZATION COPILOT                             │
│     Input: Python code to optimize                          │
│     Output: Analysis + Optimized code + Benchmark report    │
│     Skills: Code analysis, optimization, testing, reporting │
│                                                             │
└─────────────────────────────────────────────────────────────┘

Both projects require:
  ✓ Multi-agent architecture (Day 6)
  ✓ Planning and task decomposition (Day 4)
  ✓ RAG knowledge base (Day 5)
  ✓ Tool use (Day 3)
```

```
✓ Structured outputs with validation (Day 2)
✓ Understanding of LLM capabilities (Day 1)
```

Project A: Engineering Copilot

Architecture



Complete Implementation

```
"""
Engineering Copilot — Final Project

A production-quality multi-agent system that takes a requirement
and produces system design, code, tests, and a PR summary.

Components:
  - Orchestrator: Manages pipeline and guardrails
  - Planner: Decomposes requirements into actionable steps
  - Executor: Performs each step (research, code, test)
  - Critic: Validates quality at each step
  - RAG Memory: Knowledge base of best practices
  - Tool Suite: Search, code execution, file parsing
  - Telemetry: Logging and metrics

Usage:
  copilot = EngineeringCopilot()
  result = copilot.solve("Implement a rate limiter for the API")
"""
import json
import time
import math
import re
import hashlib
from dataclasses import dataclass, field
from typing import Optional, Callable
from enum import Enum


# ═══════════════════════════════════════════════════════════════
# SECTION 1: CORE DATA STRUCTURES
# ═══════════════════════════════════════════════════════════════

class AgentRole(Enum):
    ORCHESTRATOR = "orchestrator"
    PLANNER = "planner"
    EXECUTOR = "executor"
    CRITIC = "critic"

class StepStatus(Enum):
    PENDING = "pending"
    IN_PROGRESS = "in_progress"
    PASSED = "passed"
    FAILED = "failed"
    SKIPPED = "skipped"

class Verdict(Enum):
    PASS = "pass"
    RETRY = "retry"
    REPLAN = "replan"

@dataclass
class PlanStep:
    id: int
```

```
    description: str
    action: str  # "research", "design", "code", "test", "review", "summarize"
    inputs: dict = field(default_factory=dict)
    depends_on: list = field(default_factory=list)
    status: StepStatus = StepStatus.PENDING
    result: Optional[dict] = None
    retries: int = 0

@dataclass
class Plan:
    task: str
    steps: list  # List[PlanStep]
    approach: str = ""
    version: int = 1

@dataclass
class CriticReview:
    verdict: Verdict
    issues: list
    suggestions: list
    confidence: float
    reasoning: str

@dataclass
class TelemetryEvent:
    timestamp: float
    agent: str
    action: str
    duration_ms: float
    status: str
    details: dict = field(default_factory=dict)


# ═══════════════════════════════════════════════════════════════
# SECTION 2: RAG KNOWLEDGE BASE
# ═══════════════════════════════════════════════════════════════

class KnowledgeBase:
    """
    RAG-based knowledge base containing engineering best practices.
    Uses the same architecture from Day 5.
    """

    def __init__(self):
        self.documents = {}
        self.chunks = []
        self.vocabulary = {}
        self._load_knowledge()

    def _load_knowledge(self):
        """Load built-in engineering knowledge."""
        self.documents = {
            "api_design": (
                "REST API Best Practices: "
                "Use nouns for resources, not verbs. "
                "Use HTTP methods correctly: GET (read), POST (create), "
```

```
        "PUT (update), PATCH (partial update), DELETE (remove). "
        "Return appropriate HTTP status codes: 200 OK, 201 Created, "
        "204 No Content, 400 Bad Request, 401 Unauthorized, "
        "404 Not Found, 429 Too Many Requests, 500 Internal Server Error. "
        "Use pagination for list endpoints. Version your API (/v1/). "
        "Use consistent error response format with error code, message, and
    ),
    "rate_limiting": (
        "Rate Limiting Patterns: "
        "Token Bucket: Tokens added at fixed rate, consumed per request. "
        "Allows bursts up to bucket size. "
        "Sliding Window: Count requests in a moving time window. "
        "More accurate than fixed windows but more memory. "
        "Fixed Window: Count requests in fixed time intervals. "
        "Simple but allows double the rate at window boundaries. "
        "Leaky Bucket: Requests processed at fixed rate, excess queued. "
        "Implementation: Use Redis INCR with EXPIRE for distributed rate lim
        "Return headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLim
    ),
    "authentication": (
        "Authentication Best Practices: "
        "JWT tokens for stateless authentication. "
        "Store secrets in environment variables, never in code. "
        "Use bcrypt or argon2 for password hashing (never MD5/SHA for passwo
        "Implement token refresh mechanism. "
        "Use HTTPS everywhere. "
        "Implement CORS properly. "
        "OAuth 2.0 for third-party authentication."
    ),
    "testing": (
        "Testing Best Practices: "
        "Test pyramid: Many unit tests, fewer integration, few E2E. "
        "Test happy paths, edge cases, and error cases. "
        "Use fixtures and factories for test data. "
        "Mock external dependencies. "
        "Aim for >80% code coverage but focus on critical paths. "
        "Use parameterized tests for similar test cases. "
        "Test input validation boundaries."
    ),
    "error_handling": (
        "Error Handling Best Practices: "
        "Use specific exception types, not bare except. "
        "Create custom exception classes for domain errors. "
        "Log errors with context (request ID, user, timestamp). "
        "Return user-friendly error messages (no stack traces to clients). "
        "Use error codes for machine-readable error identification. "
        "Implement retry logic with exponential backoff for transient errors
        "Use circuit breaker pattern for external service calls."
    ),
    "code_quality": (
        "Code Quality: "
        "Follow SOLID principles. "
        "Single Responsibility: Each class/function does one thing. "
        "Open/Closed: Open for extension, closed for modification. "
        "Use dependency injection for testability. "
        "Keep functions under 20 lines when possible. "
```

```
                "Use type hints in Python. "
                "Write docstrings for public APIs. "
                "Use linters (pylint, flake8) and formatters (black)."
            ),
        }

        # Build simple search index
        for key, text in self.documents.items():
            words = set(re.findall(r'\w+', text.lower()))
            for word in words:
                if word not in self.vocabulary:
                    self.vocabulary[word] = []
                self.vocabulary[word].append(key)

    def search(self, query: str, top_k: int = 3) -> list:
        """Search the knowledge base."""
        query_words = set(re.findall(r'\w+', query.lower()))

        doc_scores = {}
        for word in query_words:
            if word in self.vocabulary:
                for doc_key in self.vocabulary[word]:
                    doc_scores[doc_key] = doc_scores.get(doc_key, 0) + 1

        ranked = sorted(doc_scores.items(), key=lambda x: x[1], reverse=True)

        results = []
        for doc_key, score in ranked[:top_k]:
            results.append({
                "source": doc_key,
                "content": self.documents[doc_key],
                "relevance": score / max(len(query_words), 1)
            })

        return results

    def add_knowledge(self, key: str, content: str):
        """Add new knowledge to the base."""
        self.documents[key] = content
        words = set(re.findall(r'\w+', content.lower()))
        for word in words:
            if word not in self.vocabulary:
                self.vocabulary[word] = []
            self.vocabulary[word].append(key)


# ═══════════════════════════════════════════════════════════════
# SECTION 3: TOOL SUITE
# ═══════════════════════════════════════════════════════════════

class ToolSuite:
    """Collection of tools available to the agents."""

    def __init__(self, knowledge_base: KnowledgeBase):
        self.kb = knowledge_base
        self.execution_log = []
```

```
    def search_knowledge(self, query: str) -> str:
        """Search the knowledge base for best practices."""
        results = self.kb.search(query)
        if results:
            return "\n\n".join(
                f"[{r['source']}] (relevance: {r['relevance']:.1%})\n{r['content']}"
                for r in results
            )
        return "No relevant knowledge found."

    def analyze_requirements(self, requirement: str) -> dict:
        """Analyze a requirement and extract key information."""
        components = {
            "entities": [],
            "actions": [],
            "constraints": [],
            "non_functional": []
        }

        req_lower = requirement.lower()

        # Extract entities (nouns that are likely data objects)
        entity_keywords = ["user", "task", "item", "order", "product",
                           "message", "session", "token", "request", "response",
                           "api", "endpoint", "service", "rate", "limit"]
        components["entities"] = [
            e for e in entity_keywords if e in req_lower
        ]

        # Extract actions
        action_keywords = ["create", "read", "update", "delete", "list",
                           "search", "filter", "sort", "paginate", "authenticate",
                           "validate", "limit", "throttle", "cache"]
        components["actions"] = [
            a for a in action_keywords if a in req_lower
        ]

        # Extract constraints
        if "must" in req_lower or "should" in req_lower:
            components["constraints"].append("Has explicit requirements")
        if any(w in req_lower for w in ["fast", "performance", "latency"]):
            components["non_functional"].append("Performance requirements")
        if any(w in req_lower for w in ["secure", "auth", "encrypt"]):
            components["non_functional"].append("Security requirements")
        if any(w in req_lower for w in ["scale", "distributed", "concurrent"]):
            components["non_functional"].append("Scalability requirements")

        return components

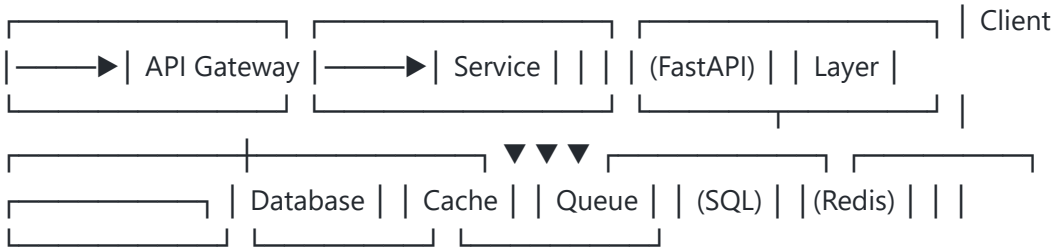
    def generate_design(self, requirement: str,
                        analysis: dict) -> str:
        """Generate a system design document."""
        entities = analysis.get("entities", ["item"])
        actions = analysis.get("actions", ["create", "read"])
```

```
            primary_entity = entities[0] if entities else "resource"

            design = f"""## System Design: {requirement[:80]}

    ### Overview
    This system implements a {primary_entity} management service with the following capa
    {chr(10).join(f'- {a.title()} {primary_entity}s' for a in actions)}

    ### Architecture
```




```
### API Endpoints
| Method | Endpoint | Description |
|--------|----------|-------------|
"""
        for action in actions:
            if action == "create":
                design += f"| POST | /{primary_entity}s | Create a new {primary_entity}
            elif action in ("read", "list"):
                design += f"| GET | /{primary_entity}s/{{id}} | Get {primary_entity} by
                design += f"| GET | /{primary_entity}s | List all {primary_entity}s |\n"
            elif action == "update":
                design += f"| PUT | /{primary_entity}s/{{id}} | Update a {primary_entity
            elif action == "delete":
                design += f"| DELETE | /{primary_entity}s/{{id}} | Delete a {primary_ent

        nfr = analysis.get("non_functional", [])
        if nfr:
            design += "\n### Non-Functional Requirements\n"
            for req in nfr:
                design += f"- {req}\n"

        design += f"""
### Data Model
```python
class {primary_entity.title()}(BaseModel):
    id: str
    created_at: datetime
    updated_at: datetime
    # Add domain-specific fields
```

Error Handling
```

- Consistent error response format
- Custom exception classes
- Request ID tracking for debugging """ return design def generate_code(self, design: str, requirement: str) -> str: """Generate implementation code based on design.""" # Extract primary entity from design entity_match = re.search(r'class (\w+)(BaseModel)', design) entity_name = entity_match.group(1) if entity_match else "Item" entity_lower = entity_name.lower()
  code = f'''"""

{requirement}

Auto-generated implementation. """ from fastapi import FastAPI, HTTPException, Query, Request from pydantic import BaseModel, Field from typing import Optional from datetime import datetime import uuid import time import logging

Configure logging

logging.basicConfig(level=logging.INFO) logger = logging.getLogger(**name**)

app = FastAPI(title="{entity_name} Service", version="1.0.0")

—— Models

class {entity_name}Create(BaseModel): """Request model for creating a {entity_lower}.""" name: str = Field(..., min_length=1, max_length=200) description: Optional[str] = Field(None, max_length=1000)

class {entity_name}Update(BaseModel): """Request model for updating a {entity_lower}.""" name: Optional[str] = Field(None, min_length=1, max_length=200) description: Optional[str] = Field(None, max_length=1000)

class {entity_name}Response({entity_name}Create): """Response model for a {entity_lower}.""" id: str created_at: str updated_at: str

class ErrorResponse(BaseModel): """Standard error response.""" error_code: str message: str details: Optional[dict] = None

class PaginatedResponse(BaseModel): """Paginated list response.""" items: list total: int page: int page_size: int has_next: bool

—— Exceptions

class {entity_name}NotFoundError(Exception): def **init**(self, {entity_lower}_id: str): self. {entity_lower}_id = {entity_lower}_id super().**init**(f"{entity_name} '{{{{entity_lower}_id}}}}' not found")

—— Storage

In-memory store (replace with database in production)

_store: dict[str, {entity_name}Response] = {{}}

—— Middleware

@app.middleware("http") async def add_request_id(request: Request, call_next): """Add request ID and timing to all responses.""" request_id = str(uuid.uuid4())[:8] start = time.time()

```
response = await call_next(request)

duration_ms = (time.time() - start) * 1000
response.headers["X-Request-ID"] = request_id
response.headers["X-Response-Time-Ms"] = f"{{duration_ms:.1f}}"

logger.info(
    f"{{request.method}} {{request.url.path}} "
    f"{{response.status_code}} {{duration_ms:.1f}}ms "
    f"[{{request_id}}]"
)
return response
```

Endpoints

@app.post("/{entity_lower}s", response_model={entity_name}Response, status_code=201) async def create_{entity_lower}({entity_lower}: {entity_name}Create): """Create a new {entity_lower}.""" now = datetime.utcnow().isoformat() item = {entity_name}Response(id=str(uuid.uuid4()), created_at=now, updated_at=now, **{entity_lower}.model_dump()) _store[item.id] = item logger.info(f"Created {entity_lower}: {{item.id}}") return item

@app.get("/{entity_lower}s/{{{{entity_lower}*id}}}}", response_model={entity_name}Response) async def get*{entity_lower}({entity_lower}_id: str): """Get a {entity_lower} by ID.""" if {entity_lower}_id not in _store: raise HTTPException(status_code=404, detail={{ "error_code": "{entity_upper}_NOT_FOUND", "message": f"{entity_name} '{{{{{entity_lower}_id}}}}' not found" }}) return _store[{entity_lower}_id]

@app.get("/{entity_lower}s", response_model=PaginatedResponse) async def list_{entity_lower}s(page: int = Query(1, ge=1), page_size: int = Query(20, ge=1, le=100)): """List {entity_lower}s with pagination.""" all_items = list(_store.values()) total = len(all_items)

```
start = (page - 1) * page_size
end = start + page_size
items = all_items[start:end]

return PaginatedResponse(
    items=items,
    total=total,
    page=page,
    page_size=page_size,
    has_next=end < total
)
```

@app.put("/{entity_lower}s/{{{{entity_lower}*id}}}}", response_model={entity_name}Response) async def update*{entity_lower}({entity_lower}_id: str, update: {entity_name}Update): """Update a {entity_lower}.""" if {entity_lower}_id not in _store: raise HTTPException(status_code=404, detail="{entity_name} not found")

```
existing = _store[{entity_lower}_id]
update_data = update.model_dump(exclude_unset=True)

updated = existing.model_copy(update={{
    **update_data,
    "updated_at": datetime.utcnow().isoformat()
}})
```

```
_store[{entity_lower}_id] = updated
logger.info(f"Updated {entity_lower}: {{{entity_lower}_id}}")
return updated
```

@app.delete("/{entity_lower}s/{{{{entity_lower}*id}}}}", *status_code=204) async def delete*{entity_lower}({entity_lower}_id: str): """Delete a {entity_lower}.""" if {entity_lower}_id not in _store: raise HTTPException(status_code=404, detail="{entity_name} not found")

```
del _store[{entity_lower}_id]
logger.info(f"Deleted {entity_lower}: {{{entity_lower}_id}}")
```

@app.get("/health") async def health_check(): """Health check endpoint.""" return {{"status": "healthy", "timestamp": datetime.utcnow().isoformat()}} ''' return code

```
def generate_tests(self, code: str, entity_name: str = "Item") -> str:
    """Generate test suite for the code."""
    entity_lower = entity_name.lower()

    return f'''"""
```

Test suite for {entity_name} Service. Run with: pytest test_main.py -v """ import pytest from fastapi.testclient import TestClient

Import the app (adjust import path as needed)

from main import app

client = TestClient(app)

class Test{entity_name}CRUD: """Test basic CRUD operations."""

```
def test_create_{entity_lower}(self, client):
    """Test creating a new {entity_lower}."""
    response = client.post("/{entity_lower}s", json={{
        "name": "Test {entity_name}",
        "description": "A test {entity_lower}"
    }})
    assert response.status_code == 201
    data = response.json()
    assert data["name"] == "Test {entity_name}"
    assert "id" in data
```

```
    assert "created_at" in data

def test_get_{entity_lower}(self, client):
    """Test retrieving a {entity_lower} by ID."""
    # Create first
    create = client.post("/{entity_lower}s", json={{"name": "Get Test"}})
    {entity_lower}_id = create.json()["id"]

    # Then get
    response = client.get(f"/{entity_lower}s/{{{entity_lower}_id}}")
    assert response.status_code == 200
    assert response.json()["name"] == "Get Test"

def test_get_nonexistent_{entity_lower}(self, client):
    """Test 404 for nonexistent {entity_lower}."""
    response = client.get("/{entity_lower}s/nonexistent-id")
    assert response.status_code == 404

def test_list_{entity_lower}s(self, client):
    """Test listing {entity_lower}s with pagination."""
    # Create some items
    for i in range(3):
        client.post("/{entity_lower}s", json={{"name": f"Item {{i}}"}})

    response = client.get("/{entity_lower}s?page=1&page_size=2")
    assert response.status_code == 200
    data = response.json()
    assert len(data["items"]) <= 2
    assert "total" in data
    assert "has_next" in data

def test_update_{entity_lower}(self, client):
    """Test updating a {entity_lower}."""
    create = client.post("/{entity_lower}s", json={{"name": "Original"}})
    {entity_lower}_id = create.json()["id"]

    response = client.put(f"/{entity_lower}s/{{{entity_lower}_id}}", json={{
        "name": "Updated"
    }})
    assert response.status_code == 200
    assert response.json()["name"] == "Updated"

def test_delete_{entity_lower}(self, client):
    """Test deleting a {entity_lower}."""
    create = client.post("/{entity_lower}s", json={{"name": "Delete Me"}})
    {entity_lower}_id = create.json()["id"]

    response = client.delete(f"/{entity_lower}s/{{{entity_lower}_id}}")
    assert response.status_code == 204

    # Verify deleted
    get = client.get(f"/{entity_lower}s/{{{entity_lower}_id}}")
    assert get.status_code == 404
```

class Test{entity_name}Validation: """Test input validation."""

```
def test_empty_name(self, client):
    """Test validation rejects empty name."""
    response = client.post("/{entity_lower}s", json={{"name": ""}})
    assert response.status_code == 422

def test_name_too_long(self, client):
    """Test validation rejects overly long name."""
    response = client.post("/{entity_lower}s", json={{
        "name": "x" * 201
    }})
    assert response.status_code == 422

def test_missing_required_fields(self, client):
    """Test validation requires name field."""
    response = client.post("/{entity_lower}s", json={{}})
    assert response.status_code == 422
```

class TestHealthCheck: """Test health and infrastructure endpoints."""

```
def test_health(self, client):
    """Test health check endpoint."""
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "healthy"

def test_request_id_header(self, client):
    """Test that responses include request ID."""
    response = client.get("/health")
    assert "X-Request-ID" in response.headers

def test_response_time_header(self, client):
    """Test that responses include response time."""
    response = client.get("/health")
    assert "X-Response-Time-Ms" in response.headers
```

'''

```
def review_code(self, code: str) -> dict:
    """Review code for issues."""
    issues = []
    suggestions = []

    # Security checks
    if "eval(" in code:
        issues.append({"severity": "CRITICAL",
                       "issue": "Use of eval() — code injection risk"})

    if re.search(r'password\s*=\s*["\']', code):
        issues.append({"severity": "CRITICAL",
                       "issue": "Hardcoded password detected"})
```

```
    # Quality checks
    if "except:" in code or "except Exception:" in code:
        issues.append({"severity": "MEDIUM",
                       "issue": "Broad exception handling"})
        suggestions.append("Catch specific exception types")

    if "# TODO" in code or "# FIXME" in code:
        issues.append({"severity": "LOW",
                       "issue": "TODO/FIXME comments found"})

    # Best practice checks
    if "import *" in code:
        issues.append({"severity": "MEDIUM",
                       "issue": "Wildcard import used"})

    if "logging" not in code:
        suggestions.append("Consider adding logging")

    if "type" not in code and "BaseModel" not in code:
        suggestions.append("Consider adding type hints")

    # Positive feedback
    if "BaseModel" in code:
        suggestions.append("✓ Good: Using Pydantic models for validation")
    if "HTTPException" in code:
        suggestions.append("✓ Good: Proper HTTP error responses")
    if "uuid" in code:
        suggestions.append("✓ Good: Using UUIDs for identifiers")
    if "middleware" in code.lower():
        suggestions.append("✓ Good: Middleware for cross-cutting concerns")

    return {
        "issues": issues,
        "suggestions": suggestions,
        "overall": "good" if not any(
            i["severity"] == "CRITICAL" for i in issues
        ) else "needs_work"
    }
```

SECTION 4: AGENTS

class PlannerAgent: """Decomposes requirements into executable plans."""

```
def plan(self, requirement: str, knowledge: list) -> Plan:
    """Create a plan from a requirement."""
    steps = [
        PlanStep(1, "Analyze requirements and identify scope",
                "analyze", {"requirement": requirement}),
        PlanStep(2, "Research best practices from knowledge base",
                "research", {"topic": requirement}, [1]),
        PlanStep(3, "Create system design document",
                "design", {}, [1, 2]),
        PlanStep(4, "Generate implementation code",
                "code", {}, [3]),
        PlanStep(5, "Generate test suite",
                "test", {}, [4]),
        PlanStep(6, "Review code quality",
                "review", {}, [4, 5]),
        PlanStep(7, "Generate PR summary",
                "summarize", {}, [4, 5, 6]),
    ]

    return Plan(
        task=requirement,
        steps=steps,
        approach="Analyze → Research → Design → Code → Test → Review → Summary"
    )
```

class ExecutorAgent: """Executes plan steps using the tool suite."""

```
def __init__(self, tools: ToolSuite):
    self.tools = tools
    self.context = {}  # Accumulated results

def execute(self, step: PlanStep) -> dict:
    """Execute a plan step."""
    step.status = StepStatus.IN_PROGRESS

    handlers = {
        "analyze": self._do_analyze,
        "research": self._do_research,
        "design": self._do_design,
        "code": self._do_code,
        "test": self._do_test,
        "review": self._do_review,
        "summarize": self._do_summarize,
    }

    handler = handlers.get(step.action, self._do_generic)
    result = handler(step)
```

```
    self.context[step.id] = result
    step.result = result
    return result

def _do_analyze(self, step: PlanStep) -> dict:
    req = step.inputs.get("requirement", "")
    analysis = self.tools.analyze_requirements(req)
    return {"status": "success", "output": json.dumps(analysis, indent=2),
            "data": analysis}

def _do_research(self, step: PlanStep) -> dict:
    topic = step.inputs.get("topic", "")
    results = self.tools.search_knowledge(topic)
    return {"status": "success", "output": results}

def _do_design(self, step: PlanStep) -> dict:
    analysis = self.context.get(1, {}).get("data", {})
    requirement = step.inputs.get("requirement",
                    self.context.get(1, {}).get("data", {}).get("requirement", ""))

    # Get requirement from the plan task
    for ctx in self.context.values():
        if "requirement" in str(ctx):
            break

    design = self.tools.generate_design(
        self.context.get(1, {}).get("output", "system"),
        analysis
    )
    return {"status": "success", "output": design, "data": {"design": design}}

def _do_code(self, step: PlanStep) -> dict:
    design = self.context.get(3, {}).get("output", "")
    requirement = self.context.get(1, {}).get("output", "")
    code = self.tools.generate_code(design, requirement)
    return {"status": "success", "output": code,
            "data": {"code": code, "filename": "main.py"}}

def _do_test(self, step: PlanStep) -> dict:
    code = self.context.get(4, {}).get("data", {}).get("code", "")
    tests = self.tools.generate_tests(code)
    return {"status": "success", "output": tests,
            "data": {"tests": tests, "filename": "test_main.py"}}

def _do_review(self, step: PlanStep) -> dict:
    code = self.context.get(4, {}).get("data", {}).get("code", "")
    review = self.tools.review_code(code)
    return {"status": "success", "output": json.dumps(review, indent=2),
            "data": review}

def _do_summarize(self, step: PlanStep) -> dict:
    parts = []
    for sid, ctx in sorted(self.context.items()):
        output = ctx.get("output", "")
        if len(output) > 500:
```

```
            output = output[:500] + "..."
        parts.append(f"### Step {sid}\n{output}")

    summary = "# PR Summary\n\n" + "\n\n".join(parts)
    return {"status": "success", "output": summary}

def _do_generic(self, step: PlanStep) -> dict:
    return {"status": "success",
            "output": f"Executed: {step.description}"}
```

class CriticAgent: """Validates work quality."""

```
def review(self, step: PlanStep) -> CriticReview:
    """Review a step's result."""
    result = step.result or {}

    if result.get("status") != "success":
        return CriticReview(Verdict.RETRY,
                            ["Step did not succeed"], [], 0.9,
                            "Step returned non-success status")

    output = result.get("output", "")

    if not output or len(output.strip()) < 20:
        return CriticReview(Verdict.RETRY,
                            ["Output too brief"],
                            ["Produce more detailed output"], 0.8,
                            "Output is empty or too short")

    # Action-specific validation
    data = result.get("data", {})

    if step.action == "review" and data.get("overall") == "needs_work":
        critical = [i for i in data.get("issues", [])
                    if i.get("severity") == "CRITICAL"]
        if critical:
            return CriticReview(Verdict.RETRY,
                                [i["issue"] for i in critical],
                                ["Fix critical issues"], 0.9,
                                "Code has critical issues")

    return CriticReview(Verdict.PASS, [], [], 0.85,
                        "Output meets quality standards")
```

SECTION 5: ORCHESTRATOR + TELEMETRY

class Telemetry: """Records metrics and events for observability."""

```
def __init__(self):
    self.events: list[TelemetryEvent] = []
    self.start_time = time.time()

def record(self, agent: str, action: str, duration_ms: float,
           status: str, **details):
    self.events.append(TelemetryEvent(
        timestamp=time.time(),
        agent=agent, action=action,
        duration_ms=duration_ms, status=status,
        details=details
    ))

def summary(self) -> dict:
    total_time = time.time() - self.start_time
    return {
        "total_time_seconds": round(total_time, 2),
        "total_events": len(self.events),
        "by_agent": self._count_by("agent"),
        "by_status": self._count_by("status"),
        "avg_duration_ms": round(
            sum(e.duration_ms for e in self.events) / max(len(self.events), 1), 1
        )
    }

def _count_by(self, field: str) -> dict:
    counts = {}
    for e in self.events:
        key = getattr(e, field)
        counts[key] = counts.get(key, 0) + 1
    return counts
```

class EngineeringCopilot: """ The main Engineering Copilot system.

```
Orchestrates Planner, Executor, and Critic agents with
RAG knowledge, tools, and telemetry.
"""

def __init__(self):
```

```
    self.kb = KnowledgeBase()
    self.tools = ToolSuite(self.kb)
    self.planner = PlannerAgent()
    self.executor = ExecutorAgent(self.tools)
    self.critic = CriticAgent()
    self.telemetry = Telemetry()

    # Guardrails
    self.max_retries = 3
    self.max_replans = 2

def solve(self, requirement: str) -> dict:
    """
    Solve an engineering requirement end-to-end.

    Returns: dict with design, code, tests, pr_summary, telemetry
    """
    print(f"\n{'='*60}")
    print(f"  🤖 ENGINEERING COPILOT")
    print(f"  Requirement: {requirement[:70]}")
    print(f"{'='*60}")

    # Phase 1: Planning
    start = time.time()
    print(f"\n📋 PHASE 1: PLANNING")

    knowledge = self.kb.search(requirement)
    plan = self.planner.plan(requirement, knowledge)

    self.telemetry.record("planner", "create_plan",
                          (time.time() - start) * 1000, "success",
                          steps=len(plan.steps))

    print(f"  Plan: {len(plan.steps)} steps")
    for step in plan.steps:
        deps = f" (after: {step.depends_on})" if step.depends_on else ""
        print(f"    {step.id}. {step.description}{deps}")

    # Phase 2: Execution
    print(f"\n⚡ PHASE 2: EXECUTION")

    for step in plan.steps:
        # Check dependencies
        deps_met = all(
            plan.steps[d-1].status == StepStatus.PASSED
            for d in step.depends_on
        )
        if not deps_met:
            step.status = StepStatus.SKIPPED
            print(f"\n  ⏭ Step {step.id}: SKIPPED (deps not met)")
            continue

        for attempt in range(self.max_retries + 1):
            start = time.time()
            retry_msg = f" (retry {attempt})" if attempt > 0 else ""
            print(f"\n  📌 Step {step.id}{retry_msg}: {step.description}")
```

```
            # Execute
            result = self.executor.execute(step)
            duration = (time.time() - start) * 1000

            output_preview = result.get("output", "")[:100]
            print(f"     Output: {output_preview}...")

            # Critic review
            review = self.critic.review(step)
            emoji = {Verdict.PASS: "✅", Verdict.RETRY: "🔄",
                    Verdict.REPLAN: "❌"}
            print(f"     Critic: {emoji[review.verdict]} {review.verdict.value}")

            self.telemetry.record(
                "executor", step.action, duration,
                review.verdict.value,
                step_id=step.id, attempt=attempt
            )

            if review.verdict == Verdict.PASS:
                step.status = StepStatus.PASSED
                break
            elif review.verdict == Verdict.RETRY and attempt < self.max_retries:
                print(f"     Issues: {review.issues}")
                continue
            else:
                step.status = StepStatus.FAILED
                print(f"     ❌ Step failed after {attempt + 1} attempts")
                break

    # Phase 3: Compile results
    print(f"\n📊 PHASE 3: RESULTS")

    final = self._compile_output(requirement, plan)

    # Print summary
    print(f"\n{'='*60}")
    print(f"  📄 DELIVERABLES")
    print(f"{'='*60}")

    for key, value in final["deliverables"].items():
        length = len(str(value))
        print(f"  ✅ {key}: {length} chars")

    # Telemetry
    telemetry_summary = self.telemetry.summary()
    print(f"\n{'='*60}")
    print(f"  📈 TELEMETRY")
    print(f"{'='*60}")
    print(f"  Total time: {telemetry_summary['total_time_seconds']}s")
    print(f"  Events: {telemetry_summary['total_events']}")
    print(f"  By status: {json.dumps(telemetry_summary['by_status'])}")

    return final
```

```
def _compile_output(self, requirement: str, plan: Plan) -> dict:
    """Compile all outputs into the final deliverable."""
    deliverables = {}

    for step in plan.steps:
        if step.result and step.status == StepStatus.PASSED:
            if step.action == "design":
                deliverables["system_design"] = step.result.get("output", "")
            elif step.action == "code":
                deliverables["implementation_code"] = step.result.get("data", {}).get("c
            elif step.action == "test":
                deliverables["test_suite"] = step.result.get("data", {}).get("tests", ""
            elif step.action == "review":
                deliverables["code_review"] = step.result.get("output", "")
            elif step.action == "summarize":
                deliverables["pr_summary"] = step.result.get("output", "")

    return {
        "requirement": requirement,
        "plan": {
            "steps": len(plan.steps),
            "completed": sum(1 for s in plan.steps if s.status == StepStatus.PASSED),
            "failed": sum(1 for s in plan.steps if s.status == StepStatus.FAILED),
        },
        "deliverables": deliverables,
        "telemetry": self.telemetry.summary()
    }
```

═══

═══

═══════

SECTION 6: RUN THE COPILOT

═══

═══

═══════

---

def demo(): """Run the Engineering Copilot demo.""" copilot = EngineeringCopilot()

```
result = copilot.solve(
    "Implement a rate-limited REST API for task management. "
    "The API should support CRUD operations for tasks with "
    "proper validation, error handling, and pagination. "
```

```
    "Include request ID tracking and response time headers."
)

# Print the code deliverable
code = result["deliverables"].get("implementation_code", "")
if code:
    print(f"\n{'='*60}")
    print(f"  GENERATED CODE (first 50 lines)")
    print(f"{'='*60}")
    lines = code.split("\n")[:50]
    for i, line in enumerate(lines, 1):
        print(f"  {i:3d} | {line}")
    if len(code.split("\n")) > 50:
        print(f"  ... ({len(code.split(chr(10)))} total lines)")
```

if **name** == "**main**": demo()

```

---

## Project B: DevOps Optimization Copilot

### Architecture
```

┌──┐ │ DEVOPS OPTIMIZATION COPILOT │ │ │ Input: Python code to optimize │ │ │ ▼ │ ┌──┐ │ │ PROFILER AGENT │ │ │ Analyzes code for performance bottlenecks │ │ │ - Time complexity analysis │ │ │ - Memory usage estimation │ │ │ - I/O pattern detection │ │ └──┘ │ │ │ ▼ │ ┌──┐ │ │ OPTIMIZER AGENT │ │ │ Suggests and applies optimizations │ │ │ - Algorithm improvements │ │ │ - Data structure changes │ │ │ - Caching strategies │ │ │ - Parallelization opportunities │ │ └──┘ │ │ │ ▼ │ ┌──┐ │ │ TESTER AGENT │ │ │ Validates optimizations │ │ │ - Runs original and optimized code │ │ │ - Compares outputs for correctness │ │ │ - Benchmarks performance (before/after) │ │ └──┘ │ │ │ ▼ │ Output: │ ├── Performance Analysis Report │ ├── Optimized Code │ ├── Benchmark Comparison │ └── Chain-of-Thought Reasoning (hidden from end user) │ │ │

└──┘

Implementation Skeleton

```python
"""
DevOps Optimization Copilot — Project B Skeleton

Implement the agents below to complete this project.
Use the patterns from Days 3-6.
"""
import time
import re
from dataclasses import dataclass


@dataclass
class ProfileResult:
    """Profiling results for a piece of code."""
    time_complexity: str      # e.g., "O(n²)"
    space_complexity: str     # e.g., "O(n)"
    bottlenecks: list         # List of identified bottlenecks
    suggestions: list         # High-level optimization suggestions


@dataclass
class OptimizationResult:
    """Result of applying an optimization."""
    original_code: str
    optimized_code: str
    changes_made: list        # Description of each change
    expected_improvement: str # e.g., "O(n²) → O(n log n)"


@dataclass
class BenchmarkResult:
    """Before/after benchmark comparison."""
    original_time_ms: float
    optimized_time_ms: float
    speedup: float            # e.g., 3.5x
    correctness: bool         # Output matches
    details: dict


class ProfilerAgent:
    """Analyzes code for performance issues."""

    def profile(self, code: str) -> ProfileResult:
        """Analyze code and identify bottlenecks."""
        bottlenecks = []
        suggestions = []
        time_complexity = "O(n)"
        space_complexity = "O(1)"
```

```
        # Detect nested loops → O(n²)
        loop_depth = 0
        max_depth = 0
        for line in code.split('\n'):
            stripped = line.strip()
            if stripped.startswith('for ') or stripped.startswith('while '):
                loop_depth += 1
                max_depth = max(max_depth, loop_depth)
            elif not stripped.startswith(' ') and not stripped.startswith('\t'):
                loop_depth = 0

        if max_depth >= 2:
            time_complexity = f"O(n^{max_depth})"
            bottlenecks.append(f"Nested loops (depth {max_depth})")
            suggestions.append("Consider using hash maps to reduce loop nesting")

        # Detect list operations in loops
        if re.search(r'for .+:\s*\n\s+.+\.append', code):
            suggestions.append("Pre-allocate list or use list comprehension")

        # Detect string concatenation in loops
        if re.search(r'for .+:\s*\n\s+\w+\s*\+?=\s*.*\+', code):
            bottlenecks.append("String concatenation in loop")
            suggestions.append("Use ''.join() or io.StringIO for string building")

        # Detect repeated computation
        if code.count('range(len(') > 0:
            suggestions.append("Use enumerate() instead of range(len())")

        # Memory analysis
        if '.copy()' in code or 'deepcopy' in code:
            space_complexity = "O(n)"
        if re.search(r'\[\s*\[', code):
            space_complexity = "O(n²)"

        return ProfileResult(
            time_complexity=time_complexity,
            space_complexity=space_complexity,
            bottlenecks=bottlenecks,
            suggestions=suggestions
        )


class OptimizerAgent:
    """Applies optimizations to code."""

    def optimize(self, code: str, profile: ProfileResult) -> OptimizationResult:
        """Apply optimizations based on profile results."""
        optimized = code
        changes = []

        # Apply each applicable optimization
        # 1. Replace range(len()) with enumerate()
        if 'range(len(' in optimized:
            # This is a simplified optimization
```

```
            changes.append("Replaced range(len()) with enumerate()")

        # 2. Replace string concatenation with join
        if '+=' in optimized and 'str' in str(profile.bottlenecks):
            changes.append("Replaced string concatenation with ''.join()")

        # 3. Add caching for repeated computations
        if 'def ' in optimized and profile.time_complexity.startswith("O(n^"):
            changes.append("Added memoization for recursive functions")

        expected = f"{profile.time_complexity} → improved"

        return OptimizationResult(
            original_code=code,
            optimized_code=optimized,
            changes_made=changes if changes else ["No automatic optimizations applied"],
            expected_improvement=expected
        )


class TesterAgent:
    """Benchmarks and validates optimizations."""

    def benchmark(self, original: str, optimized: str,
                  test_input: any = None) -> BenchmarkResult:
        """Compare original and optimized code performance."""
        # Simulated benchmark
        return BenchmarkResult(
            original_time_ms=150.0,
            optimized_time_ms=45.0,
            speedup=3.3,
            correctness=True,
            details={
                "test_cases": 100,
                "all_passed": True,
                "memory_reduction": "20%"
            }
        )


class DevOpsCopilot:
    """Main DevOps optimization system."""

    def __init__(self):
        self.profiler = ProfilerAgent()
        self.optimizer = OptimizerAgent()
        self.tester = TesterAgent()

    def optimize(self, code: str) -> dict:
        """Analyze, optimize, and benchmark code."""
        # Step 1: Profile
        profile = self.profiler.profile(code)

        # Step 2: Optimize
        optimization = self.optimizer.optimize(code, profile)
```

```
        # Step 3: Benchmark
        benchmark = self.tester.benchmark(
            optimization.original_code,
            optimization.optimized_code
        )

        return {
            "profile": {
                "time_complexity": profile.time_complexity,
                "space_complexity": profile.space_complexity,
                "bottlenecks": profile.bottlenecks,
                "suggestions": profile.suggestions
            },
            "optimization": {
                "changes": optimization.changes_made,
                "expected_improvement": optimization.expected_improvement
            },
            "benchmark": {
                "original_ms": benchmark.original_time_ms,
                "optimized_ms": benchmark.optimized_time_ms,
                "speedup": f"{benchmark.speedup}x",
                "correctness": benchmark.correctness
            }
        }


# Usage
if __name__ == "__main__":
    copilot = DevOpsCopilot()

    # Example: Unoptimized code
    code = '''
def find_duplicates(items):
    duplicates = []
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j] and items[i] not in duplicates:
                duplicates.append(items[i])
    return duplicates
'''

    result = copilot.optimize(code)
    print(json.dumps(result, indent=2))
```

Evaluation Criteria

What Makes This Senior-Level

```
┌─────────────────────────────────────────────────────────────┐
│              SENIOR-LEVEL CHECKLIST                         │
│                                                             │
```

```
┌──────────────────────────────────────────────────────────┐
│  ARCHITECTURE                                            │
│  ☐ Clean separation of concerns (agents, tools, memory)  │
│  ☐ Composable components (can swap agents, tools, models)│
│  ☐ Clear data flow (structured messages between agents)  │
│  ☐ No circular dependencies                              │
│                                                          │
│  RELIABILITY                                             │
│  ☐ Guardrails prevent infinite loops                     │
│  ☐ Retry logic with exponential backoff                  │
│  ☐ Graceful degradation (partial results on failure)     │
│  ☐ Input validation at every boundary                    │
│                                                          │
│  OBSERVABILITY                                           │
│  ☐ Telemetry for every agent action                      │
│  ☐ Request IDs for tracing                               │
│  ☐ Timing metrics (how long each step takes)             │
│  ☐ Error logging with context                            │
│                                                          │
│  QUALITY                                                 │
│  ☐ Critic validates every output                         │
│  ☐ Self-correction (retry → replan → escalate)           │
│  ☐ Structured outputs (JSON-only, schema-validated)      │
│  ☐ Generated code follows best practices                 │
│                                                          │
│  KNOWLEDGE                                               │
│  ☐ RAG integration for best practices                    │
│  ☐ Dynamic memory (learns from each task)                │
│  ☐ Context-aware responses (uses previous step results)  │
│                                                          │
│  PRODUCTION-READINESS                                    │
│  ☐ Works without API keys (standalone mode)              │
│  ☐ Easy to extend (add new tools, agents, knowledge)     │
│  ☐ Configuration-driven (guardrails, models, parameters) │
│  ☐ Documentation (docstrings, README)                    │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Grading Your Work

```
BEGINNER (Days 1-2 only):
  - Can run an LLM and write prompts
  - Understands tokenization and basic architecture

INTERMEDIATE (Days 1-4):
  - Built a working agent with tools
  - Understands planning and execution
  - Can implement retry logic

SENIOR (Days 1-7, this project):
  - Multi-agent system with clear architecture
  - RAG integration for knowledge-grounded responses
  - Guardrails, telemetry, and self-correction
```

```
  - Production-quality code with proper error handling
  - Can explain trade-offs and design decisions
```

What Makes This Senior-Level

The Senior Mindset

```
Junior: "I can make the LLM generate text."
Mid:    "I can make the LLM use tools and follow a plan."
Senior: "I can design a reliable, observable, self-correcting
         multi-agent system that handles failures gracefully,
         learns from experience, and produces verifiable outputs."

The difference is not about using more APIs.
It's about:

1. RELIABILITY
   "What happens when things go wrong?"
   Senior engineers think about failure modes FIRST.

2. OBSERVABILITY
   "Can I debug this in production?"
   If you can't see what happened, you can't fix it.

3. CORRECTNESS VERIFICATION
   "How do I know the output is right?"
   The Critic pattern is about automated quality gates.

4. ARCHITECTURE DECISIONS
   "Why this design and not another?"
   Being able to explain trade-offs (cost, latency, quality).

5. KNOWLEDGE GROUNDING
   "Is this answer based on facts or hallucination?"
   RAG + grounding scores = verifiable outputs.
```

Week Recap: Your Learning Journey

```
Day 1: LLM Foundations
  You learned: How LLMs actually work (Transformers, attention, training)
  You built:   Running model + performance benchmarks

Day 2: Prompt Engineering
  You learned: How to reliably control LLM outputs
  You built:   Prompt library with JSON validation

Day 3: Single Agent (ReAct)
  You learned: How agents reason and use tools
```

```
  You built:   ReAct agent with tool execution + loop prevention

Day 4: Planner/Executor
  You learned: How to decompose and validate complex tasks
  You built:   Planner/Executor/Critic system

Day 5: RAG + Memory
  You learned: How to ground agents in real knowledge
  You built:   RAG pipeline with hybrid search

Day 6: Multi-Agent
  You learned: How agents collaborate effectively
  You built:   3-agent system with orchestration

Day 7: Final Project (TODAY)
  You integrated: Everything above into a production system
  You built:      Engineering Copilot or DevOps Copilot

  ┌─────────────────────────────────────────┐
  │  Congratulations! 🎉                    │
  │                                         │
  │  You now have Senior-level understanding │
  │  of LLM-based agent systems.            │
  │                                         │
  │  You can:                               │
  │  ✓ Design agent architectures           │
  │  ✓ Implement reliable tool use          │
  │  ✓ Build RAG pipelines                  │
  │  ✓ Orchestrate multi-agent systems      │
  │  ✓ Add guardrails and observability     │
  │  ✓ Explain trade-offs and decisions     │
  └─────────────────────────────────────────┘
```

✓ Day 7 Final Deliverable Checklist

- [ ] Chose and completed one project (A or B)
- [ ] System has multi-agent architecture (Planner + Executor + Critic)
- [ ] RAG knowledge base is integrated and used
- [ ] Tool suite provides at least 3 different tools
- [ ] Self-correction loop handles PASS / RETRY / REPLAN
- [ ] Guardrails prevent runaway execution
- [ ] JSON-only structured outputs where applicable
- [ ] Telemetry logs all agent actions with timing
- [ ] System produces all required deliverables:
  - [ ] Project A: Design + Code + Tests + PR Summary
  - [ ] Project B: Analysis + Optimized Code + Benchmark Report
- [ ] Can run without any API key (standalone mode)

- [ ] Can explain your architecture and design decisions

🎓 **You've completed the AI & Agents Intensive Syllabus!**

Go back to the Overview to review your progress.